

Order

Testing Guide — Pilot End-to-End

QR-first restaurant & bar management SaaS

5 frontend apps · Express + Socket.IO API · PostgreSQL (Neon)

Razorpay test mode · Optional Cloudinary

Covers Phases 1–5: customer ordering, kitchen, POS, billing, payments, analytics, staff RBAC, audit logs, multi-language menu, happy-hour pricing, recommendations.

Generated: 2026-05-24

Table of Contents

1. System overview & topology
2. Setup & prerequisites
3. URLs and ports
4. Seeded data & user accounts
5. Role / permission matrix
6. Customer app — pages & actions
7. Admin app — pages & actions
8. Kitchen app — pages & actions
9. POS app — pages & actions
10. API reference (by module)
11. Realtime / Socket.IO events
12. End-to-end test scenarios (1–17)
13. Test data reset & cleanup
14. Troubleshooting

Where you see a `code snippet`, it's an exact value you can paste into a browser, terminal, or API client.

1. System overview & topology

QOrder is a multi-tenant SaaS for restaurants, cafes, and bars built around a QR-code customer ordering flow. There are five user-facing apps and one API; all share a Postgres database (Neon) and broadcast realtime updates via Socket.IO.

Apps

App	Purpose	Tech
API	REST + Socket.IO. Auth, tenancy, menu, orders, payments, analytics.	Node 22, Express 4, Drizzle ORM, Socket.IO
Customer	QR-scanned by diners. Menu ' cart ' place order ' track ' pay.	Next.js 15 SSR, Tailwind 4
Admin	Owner / manager control panel. Menu, tables, staff, analytics, audit.	Next.js 15 CSR, Recharts
Kitchen	KDS board for cooks. Tap to advance item state.	Next.js 15 PWA
POS	Waiter / cashier workstation. Floor view, order taking, billing, settlement.	Next.js 15 tablet-first PWA

Data flow at a glance

- Diner scans the QR sticker ' Customer app SSR-renders the menu (d 1.5 s first paint).
- Customer places order ' API writes to Postgres in a transaction ' Socket.IO broadcasts to kitchen + POS rooms.
- Kitchen marks items ready ' API updates ' POS sees "Ready to serve" live.
- Customer taps "Pay now" or cashier marks cash ' status flips to completed, table moves to cleaning.
- Admin sees revenue, popular dishes, peak hours update in real time on the overview page.

Multi-tenancy

Every business row carries a `tenant_id` and every operator query is scoped by the JWT's tenant. Customer requests are scoped by a signed QR token. The pilot deploys as one tenant ("Mango Tree") but the schema is multi-tenant from day one.

2. Setup & prerequisites

Prerequisites

- Node.js 22 (or newer) installed
- pnpm 9.15+ (install via `npm install -g pnpm`)
- A Postgres connection string. Neon is set up — see `.env` (gitignored).

Environment variables

All env values live in `.env` at the repo root. The required keys:

```
NODE_ENV=development
PORT=4000

DATABASE_URL=postgresql://...neon.tech/neondb?sslmode=require

REDIS_URL=redis://localhost:6379          # optional

JWT_SECRET=<48+ random chars>
JWT_ACCESS_EXPIRES_IN=8h
JWT_REFRESH_EXPIRES_IN=8h
QR_TOKEN_SECRET=<48+ random chars>

ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3001,http://localhost:3002,http://localhost:3003

# Cloudinary (optional – disables direct uploads if blank)
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# Razorpay (test mode keys already populated)
RAZORPAY_KEY_ID=rzp_test...
RAZORPAY_KEY_SECRET=...
RAZORPAY_WEBHOOK_SECRET=test_razorpay

NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_WS_URL=http://localhost:4000
```

First-time setup

1. `pnpm install` — installs all workspace dependencies.
2. `pnpm --filter @qrder/db db:migrate` — applies all pending migrations to Neon (idempotent).
3. `pnpm --filter @qrder/db db:seed` — seeds tenant "Mango Tree", branch Indiranagar, 12 tables, 4 kitchen stations, and the owner staff account.

Starting all 5 apps

```
pnpm dev
```

Turborepo runs all apps in parallel. After ~5 seconds:

- API on :4000
- Customer on :3000
- Admin on :3001
- Kitchen on :3002
- POS on :3003

For the QR PNG download (Admin 'Tables' QR PNG) to embed the correct domain in the QR code, set `CUSTOMER_APP_URL=http://localhost:3000` in the API's env if the default doesn't match.

3. URLs and ports

Service	Local URL	Purpose
API	http://localhost:4000	REST + Socket.IO + webhooks
Health check	http://localhost:4000/health	Liveness probe
DB health	http://localhost:4000/health/db	DB connectivity check
Customer	http://localhost:3000	Marketing landing
Customer (QR)	http://localhost:3000/q/<token>	Menu + cart + tracking
Admin login	http://localhost:3001/login	Owner / manager control panel
Kitchen login	http://localhost:3002/login	KDS board sign-in
POS login	http://localhost:3003/login	Waiter / cashier sign-in

Notes

- QR tokens come from Admin ' Tables ' Rotate or the QR PNG download.
- All operator apps share the same auth cookie domain (localhost), so the refresh-token cookie issued from any of them is honored by all.
- Each operator app stores its access token in its own localStorage key: `qrder-auth` (admin), `qrder-kds-auth` (kitchen), `qrder-pos-auth` (POS).

4. Seeded data & user accounts

The seed script creates

- 1 tenant: "Mango Tree"
- 1 branch: Indiranagar (Bengaluru)
- 4 kitchen stations: Tandoor, Grill, Bar, Dessert
- 12 tables: T01–T12
- 1 super_admin staff (see below)

Default super-admin login

Field	Value
Email	owner@mango.test
Password	Owner@123
Role	super_admin (all permissions)
Branch assignment	All branches (branchId = null)

Change this password before any production use. Create new staff with scoped roles via Admin ' Staff for daily operations.

Creating test accounts for other roles

Sign into Admin ' Staff + New staff. Recommended test set:

Role	Test email	Password	Use for
manager	manager@mango.test	Manager@1	Admin login (no super-admin actions)
cashier	cashier@mango.test	Cashier@1	POS settle / refund
waiter	waiter@mango.test	Waiter@1	POS floor / table workspace
kitchen	cook@mango.test	Cook@1	Kitchen Display System
bar	bar@mango.test	Bar@1	KDS filtered to bar station

5. Role / permission matrix

Every route is gated by Express middleware (`requirePerm( ... )`) that checks the JWT's permission scopes. The roles you can assign in Admin ' Staff map to these scopes:

Permission	super_ad-min	manager	cashier	waiter	kitchen	bar
menu:read						
menu:write			—	—	—	—
orders:read						
orders:write					—	—
orders:status						
payments:read				—	—	—
payments:write				—	—	—
tables:read					—	—
tables:write			—		—	—
staff:write		(non-admin)	—	—	—	—
settings:write			—	—	—	—
analytics:read			—	—	—	—
kitchen:operate		—	—	—		

What each app gates on

- Admin — opens for any role with a valid JWT but pages without permission return 403 from the API. Test with each role: log in, click through every sidebar item.
- Kitchen — login requires the account to have `kitchen:operate`.
- POS — needs `orders:write` for table workspace and `payments:write` for the Settle flow.

6. Customer app (<http://localhost:3000>)

Route	What it does
/	Landing / fallback. Shown when no QR token is given.
/q/[token]	Menu screen. SSR. Branding header + category chips + item cards. Shows trending strip and customizable badge on items with modifier groups.
/q/[token]/cart	Review cart with per-line cooking notes. Modifies quantity, removes lines, places order.
/q/[token]/order/[id]	Live order tracking. Shows status timeline (Placed ' Accepted ' Preparing ' Ready ' Served). Pay-now button (Razorpay), Call waiter, Request bill, post-served feedback prompt.

Customer flow actions

- Tap a category chip — filters items.
- Tap + Add on a no-options item — adds directly to cart.
- Tap Customize on an item with modifier groups — opens picker (single / multiple based on group type), shows live price preview.
- Tap the Languages icon in the header — switches to ?locale=hi etc. Re-fetches the menu with localized names/descriptions.
- Tap sticky cart bar ' cart drawer. Add cooking notes per line. Submit order — server re-prices authoritatively (you'll see happy-hour discounts applied here).
- On the tracking page: tap Call waiter or Request bill — both create a persisted service request (POS sees it).
- Tap Pay now — opens Razorpay Checkout. Use any test card from razorpay.com/docs/payments/test-mode/. Status flips to completed on success.
- After status becomes served, the feedback form appears. Rate 1-5 + optional comment.

7. Admin app (http://localhost:3001)

Route	What it does
/login	Email + password sign-in. 8h JWT issued on success.
/overview	Today / 7-day sales, completed orders, avg ticket, tables occupied. Top sellers + status counts. Auto-refresh 60s.
/orders	Live orders table with realtime Socket.IO. Filter active-only. Accept / Mark served / Complete shortcuts inline.
/menu	Categories chips + items table. CRUD via dialogs. Toggle availability inline. Item dialog supports modifier-group attach + signed Cloudinary upload (falls back to base64 when no creds).
/menu/modifiers	Modifier groups CRUD: create group with single/multiple selection + min/max. Add options with price deltas and isDefault toggle.
/tables	Branch-scoped table grid with status pills. Create / edit / delete. Rotate QR token. Download per-table QR PNG. Bulk QR sheet PDF.
/staff	Staff CRUD with role + branch assignment. Initial password on create. Reset-password dialog. Self-deletion blocked.
/customers	Searchable customer list. Loyalty points + visit count + total spend. Click row ' drawer with order history + feedback.
/inventory	Branch-scoped inventory items with reorder-level warning. Adjust (purchase / consumption / wastage / manual) with reason-driven sign. Movement history drawer.
/analytics	Charts: revenue line, top-sellers bar, peak-hours bar, payment-methods list. Range chips (Today / 7d / 30d / 90d). CSV export.
/branches	Branch CRUD. Active card highlights. Sets active branch for tables/inventory/POS.
/feedback	Avg rating (30d) + 1–5 star distribution + reverse-chrono feed of ratings & comments.
/audit-log	Append-only log of sensitive mutations: staff, branches, discounts, refunds, pricing rules. Joined with actor staff name + IP.
/settings	Account info, tenant id. Full settings panel comes in Phase 3+ polish.

8. Kitchen app (<http://localhost:3002>)

Route	What it does
/login	Sign-in for kitchen or bar staff. Requires kitchen:operate permission.
/	KDS grid. One card per active order. Tap an item to cycle status: pending ' preparing ' ready ' served. Cards age: green (< 5 min) ' amber (5–10) ' red (> 10).

Behaviors

- New order placed in customer / POS ' KDS card slides in within ~500 ms (Socket.IO).
- Tapping the last item to ready auto-cascades the order status to ready.
- Cooking notes from the diner are shown italic under the item name.
- 15s polling backup in case the Socket connection drops.

9. POS app (http://localhost:3003)

Route	What it does
/login	Waiter / cashier sign-in. 8h JWT.
/	Floor view: live table grid with status colors (available / occupied / reserved / cleaning) + active order badge + active waiter-call pulse. Branch switcher in header.
/table/[id]	Per-table workspace. Shows current order with items + totals. Five actions: Bill (PDF), Settle (cash/UPI/card/wallet), Discount (flat or coupon), Transfer (free-table picker), Split (evenly or by-item).

Key actions

- **Start order** — opens menu drawer with category chips, search, modifier picker for customizable items.
- **Add items** — same drawer; new items join the existing order.
- **Bill** — opens the invoice PDF in a new tab.
- **Settle** — record cash / card / UPI / wallet payment. Auto-completes the order when paid amount e total.
- **Discount** — apply flat ¹ amount with reason OR enter a coupon code.
- **Transfer** — choose a free table to move the order to. Old table freed if no other active orders.
- **Split** — evenly into N shares, or cycle each item through bill indices for per-item split.
- After completion, the table moves to **cleaning**. Use **Mark available** to free it.

10. API reference

Base URL: `http://localhost:4000`. All operator routes need `Authorization: Bearer <jwt>`. Customer routes are under `/q/:token` and use the signed QR token only (no JWT).

Auth (no auth required for login/refresh)

POST	<code>/v1/auth/login</code>	body: { email, password }
POST	<code>/v1/auth/refresh</code>	cookie: <code>qrder_refresh</code>
POST	<code>/v1/auth/logout</code>	
GET	<code>/v1/auth/me</code>	requires JWT

Menu

GET	<code>/v1/menu/categories</code>	
POST	<code>/v1/menu/categories</code>	body: { name, slug, displayOrder?, isActive? }
PATCH	<code>/v1/menu/categories/:id</code>	
DELETE	<code>/v1/menu/categories/:id</code>	
GET	<code>/v1/menu/items[?categoryId=]</code>	
GET	<code>/v1/menu/items/:id</code>	
POST	<code>/v1/menu/items</code>	body: { categoryId, name, basePrice, ... }
PATCH	<code>/v1/menu/items/:id</code>	
PATCH	<code>/v1/menu/items/:id/availability</code>	body: { isAvailable }
DELETE	<code>/v1/menu/items/:id</code>	
GET	<code>/v1/menu/modifier-groups</code>	
POST	<code>/v1/menu/modifier-groups</code>	
PATCH	<code>/v1/menu/modifier-groups/:id</code>	
DELETE	<code>/v1/menu/modifier-groups/:id</code>	
POST	<code>/v1/menu/modifier-groups/:groupId/modifiers</code>	
PATCH	<code>/v1/menu/modifiers/:id</code>	
DELETE	<code>/v1/menu/modifiers/:id</code>	
PUT	<code>/v1/menu/items/:id/modifier-groups</code>	body: { groupIds: [...] }
GET	<code>/v1/menu/items/:id/modifier-groups</code>	

Branches & tables

GET	<code>/v1/branches</code>	
POST	<code>/v1/branches</code>	
PATCH	<code>/v1/branches/:id</code>	
DELETE	<code>/v1/branches/:id</code>	
GET	<code>/v1/branches/:id/qr-sheet.pdf</code>	binary PDF
GET	<code>/v1/tables?branchId=...</code>	
POST	<code>/v1/tables?branchId=...</code>	
PATCH	<code>/v1/tables/:id</code>	
PATCH	<code>/v1/tables/:id/status</code>	body: { status }

DELETE	/v1/tables/:id	
POST	/v1/tables/:id/qr	rotates the active QR token
GET	/v1/tables/:id/qr.png	binary PNG

Orders & kitchen

GET	/v1/orders[?branchId=&activeOnly=true&status=]	
GET	/v1/orders/:id	
POST	/v1/orders?branchId=&tableId=	body: { items: [{menuItemId, quantity, modifierIds}], notes? }
PATCH	/v1/orders/:id/status	body: { status }
POST	/v1/orders/:id/items	body: { items: [...] }
POST	/v1/orders/:id/transfer	body: { targetTableId }
POST	/v1/orders/:id/merge	body: { sourceOrderId }
POST	/v1/orders/:id/split	body: { mode: 'evenly' 'by-item', splitCount? assignments? }
POST	/v1/orders/:id/discount	body: { couponCode?, flatAmount?, reason? }
GET	/v1/orders/:id/invoice.pdf	binary
GET	/v1/orders/:id/kot.pdf	binary
PATCH	/v1/orders/items/:itemId/status	body: { status }
GET	/v1/kitchen/queue?branchId=...	
PATCH	/v1/kitchen/items/:itemId	body: { status }

10. API reference (continued)

Payments

GET	/v1/payments/by-order/:orderId	
POST	/v1/payments	body: { orderId, method, amount, note? }
POST	/v1/payments/:id/refund	body: { amount?, reason? }
POST	/v1/payments/webhook/razorpay	Razorpay calls this; HMAC-verified

Staff & coupons

GET	/v1/staff	
POST	/v1/staff	body: { email, name, role, branchId, password, ... }
PATCH	/v1/staff/:id	
DELETE	/v1/staff/:id	
POST	/v1/staff/:id/reset-password	body: { newPassword }
GET	/v1/coupons	
POST	/v1/coupons	
PATCH	/v1/coupons/:id	
DELETE	/v1/coupons/:id	

Analytics, customers, feedback

GET	/v1/analytics/summary?range=today 7d 30d 90d[&branchId=]	
GET	/v1/analytics/top-items?range=&limit=	
GET	/v1/analytics/peak-hours?range=	
GET	/v1/analytics/revenue-series?range=	
GET	/v1/analytics/payment-methods?range=	
GET	/v1/analytics/export.csv?range=	CSV download
GET	/v1/customers[?search=]	
GET	/v1/customers/:id	includes orders[] + feedback[]
GET	/v1/feedback?limit=	
GET	/v1/feedback/summary?days=30	

Inventory, pricing rules, service requests

GET	/v1/inventory/items?branchId=	
POST	/v1/inventory/items?branchId=	
PATCH	/v1/inventory/items/:id	
DELETE	/v1/inventory/items/:id	
POST	/v1/inventory/items/:id/movements	body: { changeQty, reason }
GET	/v1/inventory/items/:id/movements	
GET	/v1/pricing-rules	

```
POST    /v1/pricing-rules          body: { name, type, value, startTime?, endTime?, ... }
PATCH  /v1/pricing-rules/:id
DELETE  /v1/pricing-rules/:id

GET     /v1/service-requests?branchId=[&openOnly=true]
POST    /v1/service-requests/:id/acknowledge
POST    /v1/service-requests/:id/resolve
```

Misc — uploads, audit log, customer (public)

```
POST    /v1/uploads/sign          body: { folder } – signed Cloudinary params

GET     /v1/audit-logs?limit=&before=

GET     /q/:token                resolve QR 'branding + table info
GET     /q/:token/menu[?locale=hi] full menu with prices, translations, mod groups
GET     /q/:token/recommendations[?itemId=] "also ordered with" or trending
POST    /q/:token/orders          body: PlaceOrderInput
GET     /q/:token/orders/:id
POST    /q/:token/orders/:id/payment/initiate
POST    /q/:token/orders/:id/payment/verify  body: gateway response
POST    /q/:token/orders/:id/feedback  body: { rating, comment? }
POST    /q/:token/waiter-call       body: { reason }
```


11. Realtime / Socket.IO events

Connect to `http://localhost:4000` with auth payload `{ token: <jwt> }` (operator) or `{ qrToken: <qr> }` (customer). Each operator client auto-joins its branch's order, kitchen, and table rooms.

Event	Direction	When it fires
order:placed	server ' operators	A new order is created (customer or POS).
order:status_changed	server ' both	Order status transitions (placed ' accepted ' preparing ' ready ' served ' completed).
order:item_status_changed	server ' both	Individual order item moves through pending/preparing/ready.
table:status_changed	server ' operators	Table flips between available / occupied / cleaning / reserved.
table:waiter_called	server ' operators	Customer pressed Call waiter or Request bill. POS shows pulse badge.
menu:item_availability_changed	server ' operators	Manager toggled an item in/out of stock.

How to verify a realtime path manually

1. Open POS at :3003 in one window.
2. Open Kitchen at :3002 in another.
3. Open Customer QR URL in a third.
4. Place an order in Customer.
5. Within ~500 ms: POS shows the table occupied, Kitchen shows the ticket.

12. End-to-end test scenarios

Run each scenario from a clean dev start (`pnpm dev`) with all apps open. Most scenarios share the same setup phase (login + active branch); a single setup gives you 30+ minutes of testing.

Scenario 1 — Owner first login & menu setup

1. Visit `http://localhost:3001/login`. Log in as `owner@mango.test` / `Owner@123`.
2. Sidebar 'Menu' click+ Category. Create "Starters". Submit.
3. Click + Item. Fill: name "Paneer Tikka", category Starters, price 240, veg, spicy=2, prep=15 min. Save.
4. Toggle Available off and on — confirm the badge updates in-place.
5. Click into Modifier groups ' create "Size" (single, min/max 1). Add Half 150, Full 10 (default).
6. Back to Menu ' edit Paneer Tikka ' check Size group ' Save.

Expected: `GET /v1/menu/items` returns the item with `modifierGroups[]` populated.

Scenario 2 — Tables & QR codes

1. Sidebar 'Tables'. The 12 seeded tables (T01–T12) appear.
2. Pick T01 ' clickQR PNG. PNG downloads.
3. Click QR sheet in the page header. A multi-table PDF sheet downloads with one QR per table.
4. Click Rotate on a table — the old QR is deactivated; previous scans now 404.

Scenario 3 — Customer ordering (QR-first)

1. From Tables, click QR PNG for T01. Note the URL embedded inside (looks like /q/ABC...).
2. Open that URL in a fresh tab ' Customer menu loads SSR.
3. Verify branding header: "Mango Tree · Indiranagar · Table T01".
4. Tap Paneer Tikka ' modifier picker opens ' choose Half (₹50) ' Add.
5. Cart bar at the bottom shows 1 item, ₹190 (= 240 - ₹50).
6. Tap cart bar ' review ' Place order.
7. Tracking page opens. POS at :3003 and Kitchen at :3002 should light up within 500 ms.

Scenario 4 — Kitchen flow

1. Kitchen :3002 ' new card appears with a sound (in real env). Card border = green (< 5min).
2. Tap the Paneer Tikka line — status ' preparing. Order auto-flips to "preparing".
3. Tap again — status ' ready. Order auto-flips to "ready". Customer's tracking page advances.

Scenario 5 — POS settle (cash)

1. POS :3003 floor view ' tap T01.
2. Order detail shows items + totals. Press Settle.
3. Method = cash, amount auto-filled to total. Submit.
4. Order auto-completes; T01 flips to cleaning.
5. Press Mark available to free the table.

Scenario 6 — Customer Razorpay payment

1. Repeat scenario 3 but stop before the cashier settles.
2. On the customer tracking page after status served, tap Pay now.
3. Razorpay Checkout opens. Pay with a test card: 4111 1111 1111 1111, any future expiry, any CVV.
4. On success, tracking page shows " Paid · Thank you!"; admin /orders reflects "completed".

The webhook secret in your .env (test_razorpay) only verifies inline checkout signatures — real webhooks need the exact secret from your Razorpay dashboard. For local testing, the verify endpoint already completes the order without needing the webhook.

Scenario 7 — Coupon discount

1. Use a SQL client to insert a coupon (no admin UI yet) — see Test data section. Code = WELCOME10, 10% off.
2. POS table workspace with an active order 'Discount.
3. Pick Coupon ' enter WELCOME10 ' Apply. Total drops 10%.
4. Audit Log ' entry "discount" appears with the diff JSON.

Scenario 8 — Split bill

1. POS with an active order having e 2 itemsSplit.
2. Choose Evenly = 3. Expected: 2 child orders created with parentOrderId; each holds 1/3 of the total.
3. Alternatively choose By item; tap each item to cycle into Bill 1 / 2 / 3.

Scenario 9 — Transfer table

1. POS ' table with an order Transfer.
2. Pick a free table from the grid.
3. Order is reassigned. Old table freed (if no other active orders). New table flips to occupied. POS auto-navigates to floor.

Scenario 10 — Merge orders

API-only for now (no POS UI):

```
POST /v1/orders/<targetOrderId>/merge
{ "sourceOrderId": "<sourceOrderId>" }
```

Source order canceled with note "merged ' #<target>". Source's items reassigned. Source's table freed.

Scenario 11 — Refund

1. POS Settle a card or UPI payment first.
2. API call (or use the refund endpoint when wired into UI):

```
POST /v1/payments/<paymentId>/refund body: { amount?, reason? }
```

3. Payment row flips to refunded; audit log entry created.

Scenario 12 — Loyalty points

1. Customer order with phone (currently set programmatically; see Test data section).
2. Complete the order. The customer row gets floor(total/1000) points (1 pt per 10).
3. Admin ' Customers ' search by phone ' drawer shows points + spend + order history.

Scenario 13 — Multi-language menu

1. API: PATCH a menu item's translations:

```
PATCH /v1/menu/items/<id>
{ "translations": { "hi": { "name": "<@N>"description": "..." } } }
```

2. Update tenant settings.supportedLocales to include "hi" (Phase 6 UI; for now: SQL).
3. Customer app ' Languages icon ' chooseMenu@-renders.

Scenario 14 — Happy-hour pricing

1. API: create a pricing rule for everyone, 20% off Paneer Tikka, 00:00–23:59:

```
POST /v1/pricing-rules
{
  "name": "Happy Hour 20%",
  "type": "percent",
  "value": 2000,
  "menuItemIds": ["<paneer item id>"],
  "startTime": "00:00",
  "endTime": "23:59",
  "isActive": true
}
```

2. Customer menu reload ' item now shows ¹192, strike-through ¹240, "Sale" pill.
3. Order it ' server applies rule ' line total reflects ¹192.

Scenario 15 — Service requests (call waiter)

1. Customer tracking page ' tapCall waiter.
2. POS floor view ' red pulse badge appears on the corresponding table.
3. POS or API:

```
GET /v1/service-requests?branchId=<id>
POST /v1/service-requests/<id>/acknowledge
POST /v1/service-requests/<id>/resolve
```

4. Status transitions: open ' acknowledged ' resolved.

Scenario 16 — Inventory adjust

1. Admin ' Inventory. Add "Paneer" (kg, 5, reorder 1, ₹350/kg).
2. Adjust ' reason Purchase, +2 kg ' stock now 7 kg.
3. Adjust ' reason Consumption, 1.5 kg ' stock now 5.5 kg (auto-signed negative).
4. Try over-consume (e.g. 100 kg) — API rejects with "Resulting stock cannot be negative".
5. History drawer shows all movements with timestamps.

Scenario 17 — Staff management & RBAC

1. Admin ' Staff + New staff. Create Riya as waiter, branch Indiranagar, password Riya@1234.
2. Open POS at :3003 in a new browser. Log in as riya@mango.test / Riya@1234. Floor view loads.
3. Open Admin at :3001 with Riya's account ' access works but waiter-allowed pages only; staff page returns 403 on API.
4. Back as owner: Reset password for Riya ' new password. Riya's old password rejected at /v1/auth/login.
5. Audit log shows the three actions: create / reset_password / and any RBAC denials.

Scenario 18 — Analytics & CSV export

1. After running scenarios 3–7, Admin ' Overview shows non-zero revenue and top sellers.
2. Admin ' Analytics. Range chips change all 4 charts. Toggle 7d / 30d / 90d.
3. Click csv — downloads orders-7d.csv.
4. Admin ' Feedback shows ratings + 30-day distribution after scenario 3's served orders get rated.

13. Test data reset & cleanup

Clear all orders / payments / feedback (keep menu + tables)

```
-- Run against Neon. Order matters: dependent rows first.
DELETE FROM order_item_modifiers;
DELETE FROM order_items;
DELETE FROM payments;
DELETE FROM feedback;
DELETE FROM orders;
DELETE FROM service_requests;
DELETE FROM stock_movements;
UPDATE customers SET loyalty_points = 0, total_spend = 0, visit_count = 0;
UPDATE tables SET status = 'available';
```

Full nuke (everything except the tenant & owner)

```
-- WARNING: this also wipes menu items and modifier groups.
TRUNCATE
audit_logs,
order_item_modifiers,
order_items,
payments,
feedback,
orders,
service_requests,
stock_movements,
notifications,
pricing_rules,
coupons,
inventory_items,
menu_item_modifier_groups,
modifiers,
modifier_groups,
menu_items,
menu_categories,
qr_codes
CASCADE;
UPDATE tables SET status = 'available';
```

Re-seed

```
pnpm --filter @qrder/db db:seed
```

Idempotent: it will fail if the tenant exists. Drop the tenants table or do a fresh DB if you want a true clean slate.

Insert a quick test coupon (no admin UI yet)

```
INSERT INTO coupons (tenant_id, code, type, value, min_subtotal, is_active)
SELECT id, 'WELCOME10', 'percent', 1000, 0, true FROM tenants LIMIT 1;
```

Insert tenant supported locales

```
UPDATE tenants
SET settings = settings || '{"supportedLocales":["en","hi"]}':::jsonb
WHERE slug = 'mango-tree';
```


14. Troubleshooting

"Auto-logged out on refresh"

If you start using a stale token (e.g. you ran the app days ago with a 15 min TTL, then `JWT_ACCESS_EXPIRES_IN` moved to 8h), the persisted token may still be expired. Open DevTools Console in the admin tab:

```
localStorage.removeItem('qrder-auth'); location.href = '/login';
```

"400 Bad Request: branchId is required"

Super-admin users have `branchId = null`. Branch-scoped endpoints (tables, kitchen queue, inventory) need an explicit `branchId`. The Admin / POS apps handle this via the branch switcher chip in the topbar — if you see this error, click the chip and pick a branch.

"PDF returns 500"

The PDF templates import React explicitly for the classic JSX transform used by tsx (the dev runtime). If you edit a template and hit a 500, check the API logs — common cause is a missing React import or returning `null` from JSX where React-PDF expects `<View />`.

"Razorpay payment never confirms"

The customer pay flow uses inline signature verification (no webhook needed). If you see issues:

- Check the API logs for `razorpay createOrder failed` — keys invalid.
- Check the customer's browser console for `checkout.js` errors — script blocked.
- On success, the customer's verify call should return 200; if it 400s, the signature is wrong (often a key-secret mismatch).

"Image upload always falls back to base64"

That means `CLOUDINARY_*` env vars aren't set — the API picks the NoOp storage. Fill in your Cloudinary creds in `.env` and restart the API. Direct signed uploads start working immediately, no code change.

"Socket.IO won't connect"

Check `ALLOWED_ORIGINS` includes the URL you're loading the operator app from. Default covers all dev ports (3000–3003).

Regenerating this PDF

```
pnpm --filter @qrder/api build:docs
```

The script is at `apps/api/scripts/build-testing-pdf.tsx`. Edit it and re-run to refresh the PDF.